Universidad Autónoma de Tamaulipas
Unidad Académica Multidisciplinaria Mante Centro

Nombre del trabajo:
Reporte de Practicas bloque 1

Asignatura:
Diseño Electrónico Basado en Sistemas Embebidos

Profesor:
Dr. Daniel López Piña

Alumno:
Sánchez Montes Ana Lilia

Grado: 8° **Grupo:** E

Ciudad Mante, Tamaulipas Abril 2026

Contenido

Introducción 3
Practica 1 Encender un led 4
Práctica 2. Entradas Digitales — Pulsadores y Pull-up/Pull-down 7
Práctica 3. Salidas Analógicas — PWM, Brillo de LED y Buzzer 10
Practica 4 Sensores Analógicos — ADC, LDR y Potenciómetro 14
Practica 5 Sensor de Temperatura y Humedad DHT22 17
Practica 6 Actuadores — Motor DC y Servomotor 21
Practica 7 Comunicación I2C — Sensor BMP280 y Pantalla OLED 24
Practica 8 Comunicación SPI y Proyecto Integrador Bloque I 29

Introducción

En el campo de los sistemas embebidos, el uso de plataformas de desarrollo como Arduino Uno permite adquirir habilidades prácticas en electrónica, programación y control de dispositivos físicos. Las prácticas de laboratorio presentadas en este curso están diseñadas para introducir de manera progresiva los conceptos fundamentales necesarios para el desarrollo de sistemas inteligentes y automatizados.

Inicialmente, se aborda la interacción básica con el microcontrolador mediante la ejecución del clásico programa Blink, el cual permite comprender el control de salidas digitales y el uso del monitor serial para la comunicación entre el usuario y el sistema. Posteriormente, se exploran las entradas digitales mediante el uso de pulsadores y configuraciones de resistencias pull-up y pull-down, fundamentales para garantizar lecturas estables.

A medida que avanzan las prácticas, se introducen conceptos de señales analógicas, como la modulación por ancho de pulso (PWM), que permite controlar la intensidad de dispositivos como LEDs y buzzers. Asimismo, se emplean sensores analógicos como el LDR y el potenciómetro, utilizando el convertidor analógico-digital (ADC) para interpretar señales del entorno.

En etapas intermedias, se integran sensores más complejos como el DHT22 para la medición de temperatura y humedad, así como actuadores como motores de corriente directa y servomotores, ampliando las capacidades de control del sistema. Finalmente, se introducen protocolos de comunicación como I2C y SPI, los cuales permiten la conexión con dispositivos externos como sensores ambientales y pantallas OLED, facilitando la creación de sistemas más avanzados e integrados.

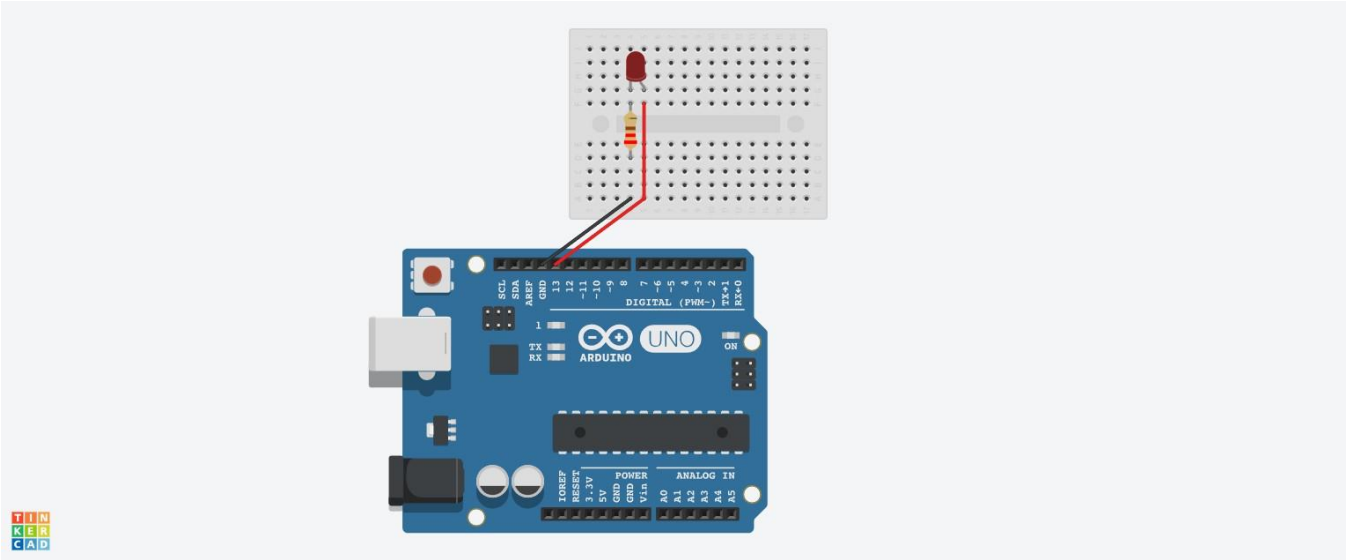
Practica 1 Encender un led

Arduino UNO o Nano (físico o simulado)

- LED rojo 5 mm
- Resistencia 220 Ω
- Protoboard + cables dupont
- Cable USB A-B para Arduino

Objetivo de la práctica

Comprender los fundamentos básicos de la programación en sistemas embebidos mediante el uso de la plataforma Arduino Uno, desarrollando un programa simple que permita el encendido y apagado intermitente de un LED (Blink), así como la utilización del monitor serial para establecer comunicación entre el microcontrolador y el usuario, con el fin de visualizar mensajes y verificar el funcionamiento del sistema.



Código

```
/* ================================================
   Lab 01 — Blink: parpadeo de LED con Arduino
   Pin de salida: 13 (LED integrado o externo)
   ================================================ */

const int PIN_LED = 13;        // Constante para el pin del LED
int contador      = 0;         // Cuenta los parpadeos

void setup() {
    // Configurar el pin como SALIDA digital
    pinMode(PIN_LED, OUTPUT);

    // Iniciar comunicacion serial a 9600 bps
    Serial.begin(9600);
```

```
    Serial.println('=== Lab 01: Blink Iniciado ===');
    Serial.println('Formato: [#N] LED -> ESTADO');
}

void loop() {
    contador++;
    // Encender el LED
    digitalWrite(PIN_LED, HIGH);
    Serial.print('[#');
    Serial.print(contador);
    Serial.println('] LED -> ENCENDIDO');
    delay(500);              // Esperar 500 ms

    // Apagar el LED
    digitalWrite(PIN_LED, LOW);
    Serial.print('[#');
    Serial.print(contador);
    Serial.println('] LED -> APAGADO');
    delay(500);              // Esperar 500 ms
}
```

VERDAD, BELLEZA, PROBIDAD

Práctica 2. Entradas Digitales — Pulsadores y Pull-up/Pull-down

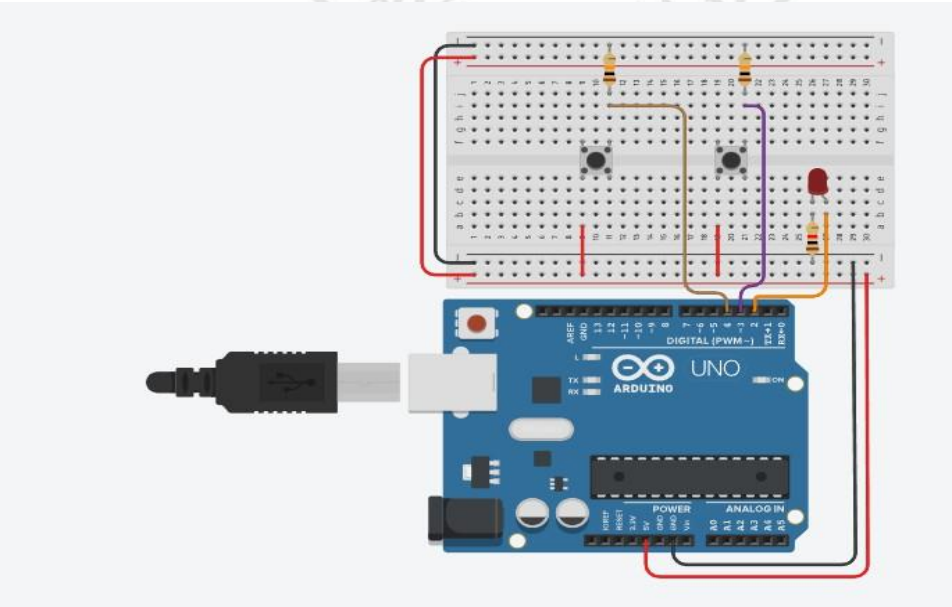
Materiales

Arduino UNO o Nano

- 2 pulsadores (push buttons)
- LED verde + resistencia 220 Ω
- Resistencias 10 kΩ (pull-down externos)
- Protoboard + cables dupont

Objetivo de la práctica

Comprender el funcionamiento de las entradas digitales en sistemas embebidos mediante el uso de pulsadores en la plataforma Arduino Uno, implementando configuraciones de resistencias pull-up y pull-down para garantizar lecturas estables, con el fin de detectar correctamente estados lógicos y controlar el comportamiento de salidas digitales en respuesta a la interacción del usuario.



Codigo

```
const int PIN_BOTON = 2;    // Entrada digital
const int PIN_LED   = 8;    // Salida digital


void setup() {
    // Entrada normal (requiere R pull-down externa)
    pinMode(PIN_BOTON, INPUT);
    pinMode(PIN_LED, OUTPUT);
    Serial.begin(9600);
    Serial.println("Lab 02 - Lectura de pulsador");
}
```

```
void loop() {
  int estado = digitalRead(PIN_BOTON);


  if (estado == HIGH) {
    digitalWrite(PIN_LED, HIGH);
    Serial.println("Boton: PRESIONADO");
  } else {
    digitalWrite(PIN_LED, LOW);
  }
  delay(50);
}
```

VERDAD, BELLEZA, PROBIDAD

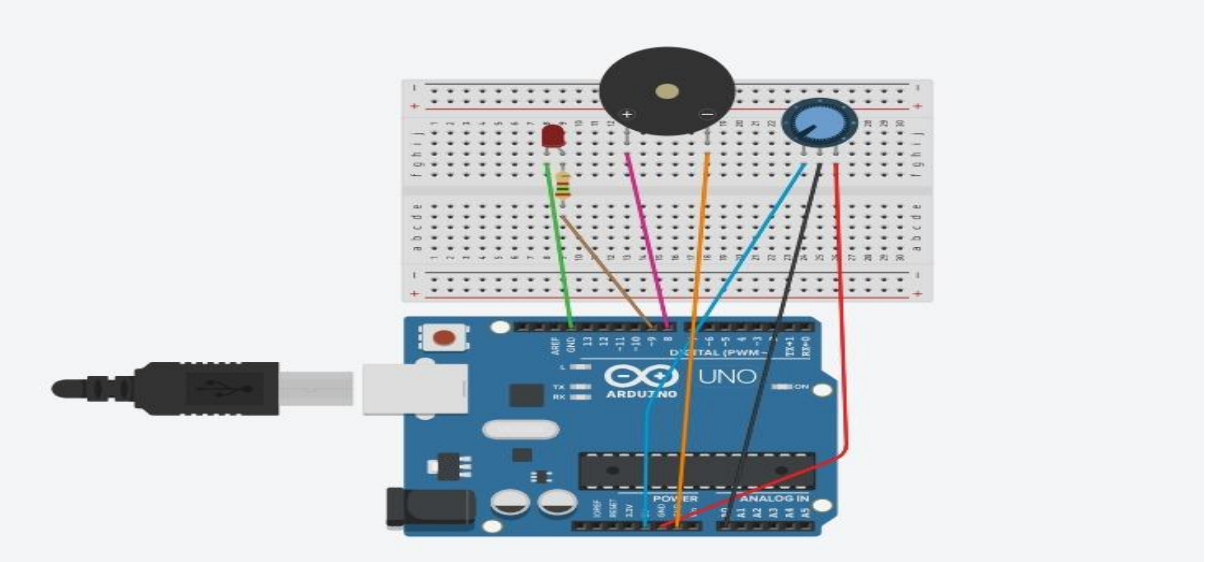
Práctica 3. Salidas Analógicas — PWM, Brillo de LED y Buzzer

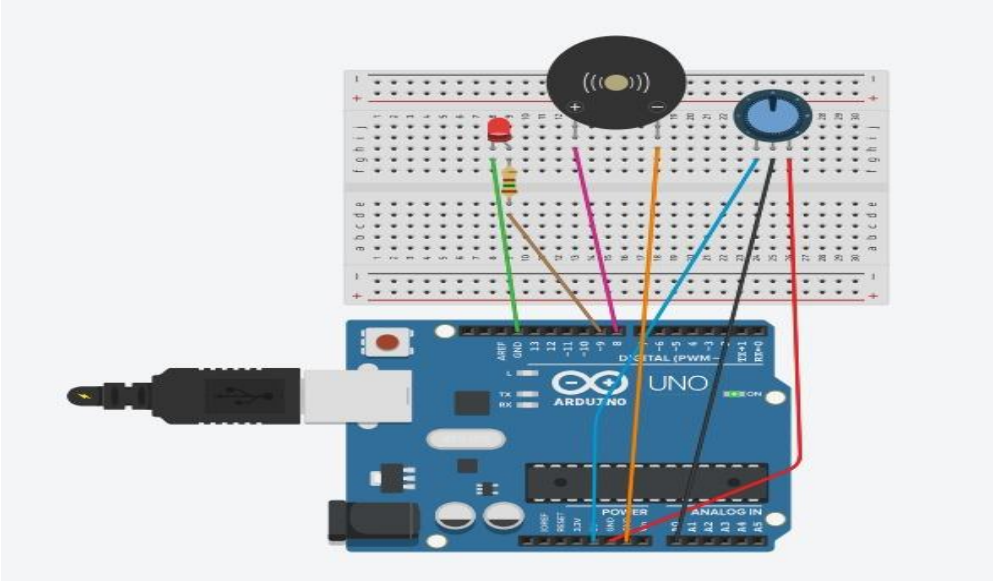
Materiales

Arduino UNO o Nano
● LED blanco o azul + resistencia 150 Ω
● Buzzer piezoeléctrico pasivo
● Potenciómetro 10 kΩ (opcional)
● Protoboard + cables

Objetivo de la práctica

Comprender el uso de las salidas analógicas en sistemas embebidos mediante la técnica de modulación por ancho de pulso (PWM) en la plataforma Arduino Uno, con el fin de controlar el brillo de un LED y la intensidad sonora de un buzzer, permitiendo variar señales de salida de manera proporcional y simular niveles analógicos a partir de señales digitales.





Código

```
int led = 9;        // LED en pin PWM
int buzzer = 8;     // Buzzer
int pot = A0;       // Potenciómetro

int valor = 0;
int brillo = 0;
int frecuencia = 0;

void setup() {
  pinMode(led, OUTPUT);
  pinMode(buzzer, OUTPUT);
  Serial.begin(9600);
```

```
}

void loop() {
  // Leer potenciómetro
  valor = analogRead(pot);

  // Convertir a brillo (0-255)
  brillo = map(valor, 0, 1023, 0, 255);

  // Convertir a frecuencia (sonido)
  frecuencia = map(valor, 0, 1023, 100, 2000);

  // LED con brillo PWM
  analogWrite(led, brillo);

  // Buzzer con tono variable
  tone(buzzer, frecuencia);
```

```
  // Mostrar en Monitor Serial
  Serial.print("Pot: ");
  Serial.print(valor);
  Serial.print(" | Brillo: ");
  Serial.print(brillo);
  Serial.print(" | Frecuencia: ");
  Serial.println(frecuencia);

  delay(200);
}
```

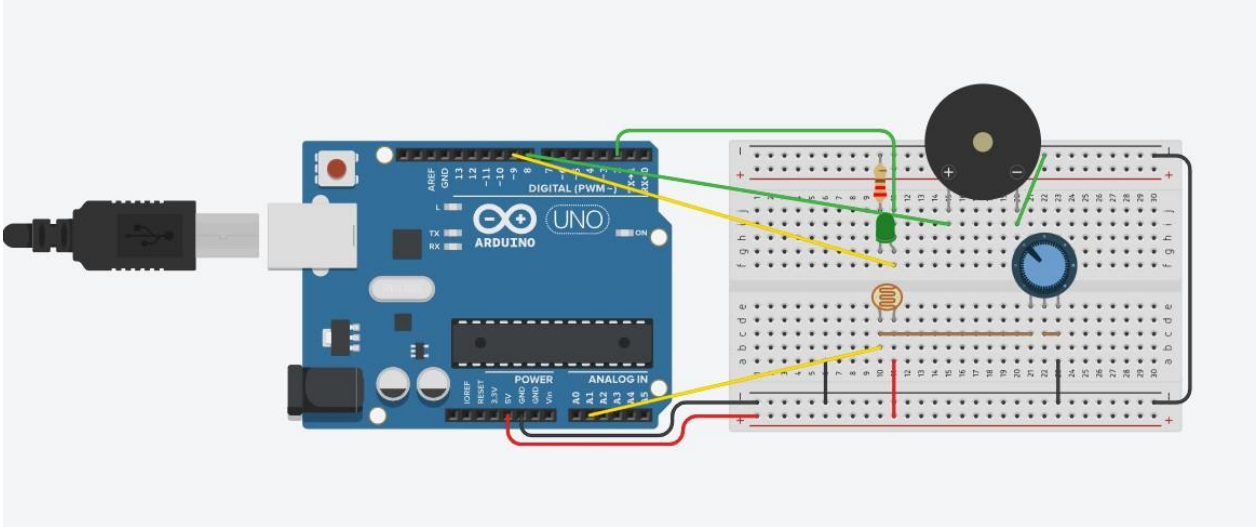
Practica 4 Sensores Analógicos — ADC, LDR y Potenciómetro

Materiales

- Arduino UNO o Nano
- Potenciómetro 10 kΩ
- LDR (fotoresistencia)
- Resistencia 10 kΩ (para divisor de voltaje LDR)
- LED + resistencia 220 Ω
- Buzzer piezoeléctrico

Objetivo de la práctica

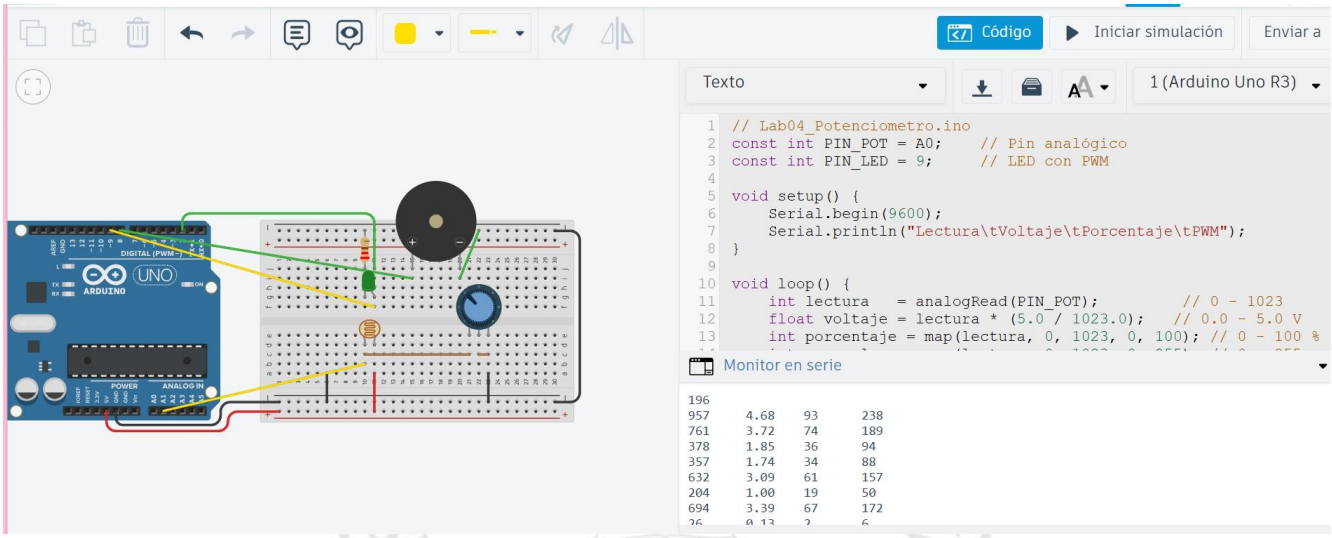
Comprender el funcionamiento de las entradas analógicas en sistemas embebidos mediante el uso del convertidor analógico-digital (ADC) en la plataforma Arduino Uno, utilizando sensores como el LDR y el potenciómetro para adquirir señales del entorno, con el fin de interpretar valores analógicos y emplearlos en el control y monitoreo de variables físicas.



Codigo 1

```
// Lab04_Potenciometro.ino
const int PIN_POT = A0;    // Pin analógico
const int PIN_LED = 9;     // LED con PWM

void setup() {
    Serial.begin(9600);
    Serial.println("Lectura\tVoltaje\tPorcentaje\tPWM");
}

void loop() {
    int lectura  = analogRead(PIN_POT);          // 0 - 1023
    float voltaje = lectura * (5.0 / 1023.0);   // 0.0 - 5.0 V
    int porcentaje = map(lectura, 0, 1023, 0, 100); // 0 - 100 %
    int pwm_val    = map(lectura, 0, 1023, 0, 255); // 0 - 255

    // Controlar brillo del LED con el potenciometro
    analogWrite(PIN_LED, pwm_val);

    // Imprimir tabla de valores
    Serial.print(lectura);     Serial.print('\t');
```

```
    Serial.print(voltaje, 2); Serial.print('\t');
    Serial.print(porcentaje); Serial.print('\t');
    Serial.println(pwm_val);

    delay(200);
}
```

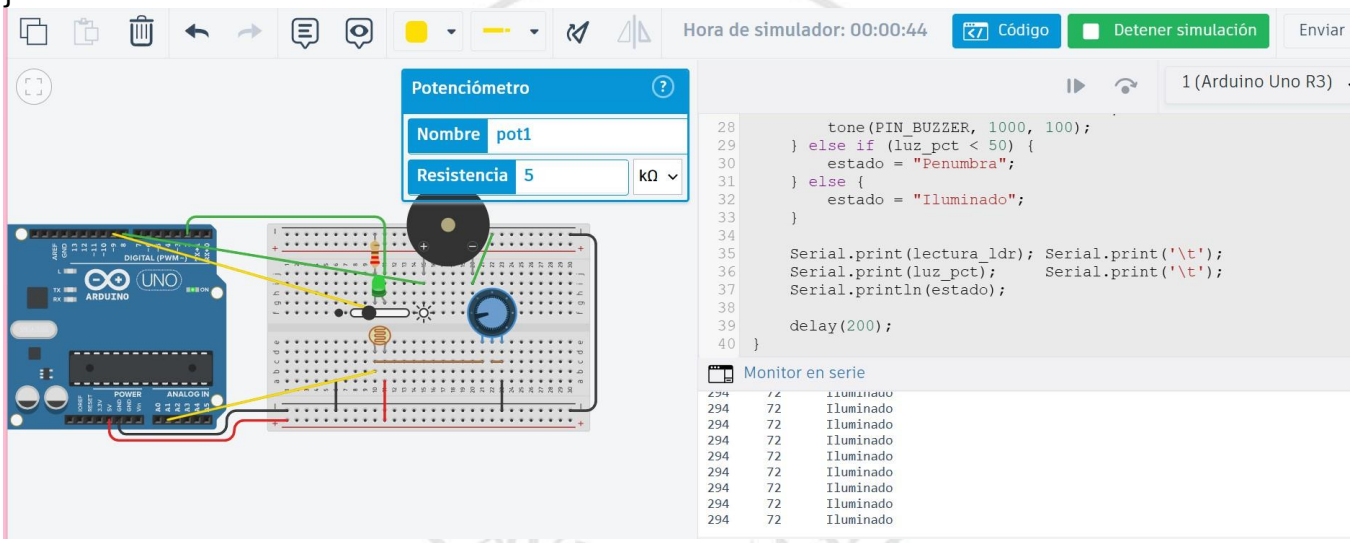


Código 2

```
// Lab04_Potenciometro.ino
const int PIN_POT = A0;    // Pin analógico
const int PIN_LED = 9;     // LED con PWM

void setup() {
    Serial.begin(9600);
    Serial.println("Lectura\tVoltaje\tPorcentaje\tPWM");
}

void loop() {
    int lectura  = analogRead(PIN_POT);          // 0 - 1023
    float voltaje = lectura * (5.0 / 1023.0);   // 0.0 - 5.0 V
    int porcentaje = map(lectura, 0, 1023, 0, 100); // 0 - 100 %
    int pwm_val    = map(lectura, 0, 1023, 0, 255); // 0 - 255
```

VERDAD, BELLEZA, PROBIDAD

```
    // Controlar brillo del LED con el potenciometro
    analogWrite(PIN_LED, pwm_val);

    // Imprimir tabla de valores
    Serial.print(lectura);     Serial.print('\t');
    Serial.print(voltaje, 2); Serial.print('\t');
    Serial.print(porcentaje); Serial.print('\t');
    Serial.println(pwm_val);

    delay(200);
}
```



Practica 5 Sensor de Temperatura y Humedad DHT22

Materiales

Arduino UNO o Nano

- Sensor DHT22 (o DHT11)
- Resistencia 10 kΩ (pull-up datos)
- LED rojo + LED verde +

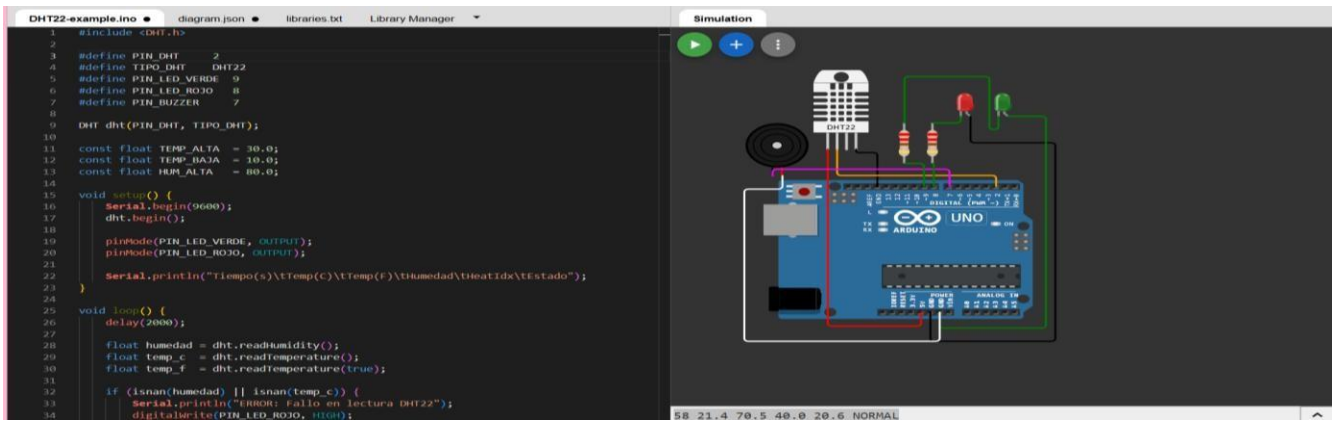
resistencias 220 Ω

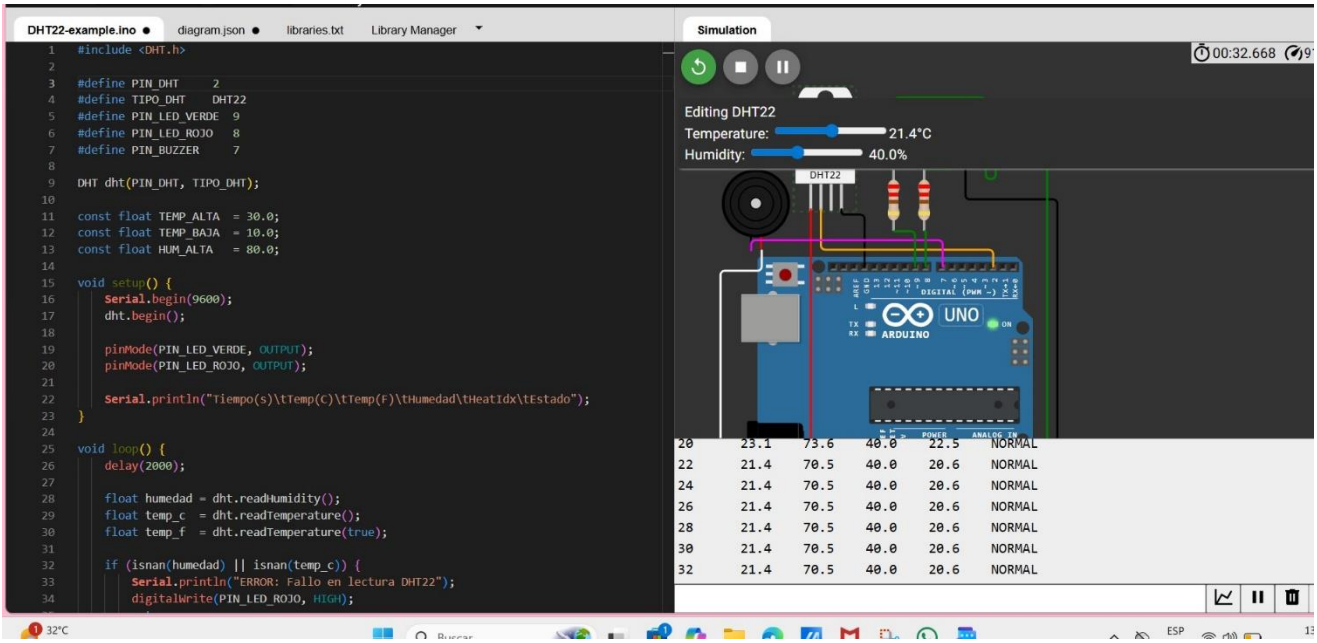
- Buzzer piezoeléctrico

VERDAD, BELLEZA, PROBIDAD

Objetivo de la práctica

Comprender el funcionamiento de las entradas analógicas en sistemas embebidos mediante el uso del convertidor analógico-digital (ADC) en la plataforma Arduino Uno, utilizando sensores como el LDR y el potenciómetro para adquirir señales del entorno, con el fin de interpretar valores analógicos y emplearlos en el control y monitoreo de variables físicas.





Codigo

```
#include <DHT.h>

#define PIN_DHT     2
#define TIPO_DHT    DHT22
#define PIN_LED_VERDE  9
#define PIN_LED_ROJO   8
#define PIN_BUZZER     7

DHT dht(PIN_DHT, TIPO_DHT);

const float TEMP_ALTA  = 30.0;
const float TEMP_BAJA  = 10.0;
const float HUM_ALTA  = 80.0;

void setup() {
   Serial.begin(9600);
   dht.begin();

   pinMode(PIN_LED_VERDE, OUTPUT);
   pinMode(PIN_LED_ROJO, OUTPUT);
```

VERDAD, BELLEZA, PROBIDAD

```
    Serial.println("Tiempo(s)\tTemp(C)\tTemp(F)\tHumedad\tHeatIdx\tEstado");
}

void loop() {
    delay(2000);

    float humedad = dht.readHumidity();
    float temp_c  = dht.readTemperature();
    float temp_f  = dht.readTemperature(true);

    if (isnan(humedad) || isnan(temp_c)) {
        Serial.println("ERROR: Fallo en lectura DHT22");
        digitalWrite(PIN_LED_ROJO, HIGH);
        return;
    }

    float heat_idx = dht.computeHeatIndex(temp_c, humedad, false);

    String estado;

    if (temp_c > TEMP_ALTA || humedad > HUM_ALTA) {
        estado = "ALERTA-CALOR";
        digitalWrite(PIN_LED_ROJO, HIGH);
        digitalWrite(PIN_LED_VERDE, LOW);
        tone(PIN_BUZZER, 2000, 200);

    } else if (temp_c < TEMP_BAJA) {
        estado = "ALERTA-FRIO";
        digitalWrite(PIN_LED_ROJO, HIGH);
        digitalWrite(PIN_LED_VERDE, LOW);
        tone(PIN_BUZZER, 500, 500);

    } else {
        estado = "NORMAL";
        digitalWrite(PIN_LED_VERDE, HIGH);
        digitalWrite(PIN_LED_ROJO, LOW);
    }

    Serial.print(millis()/1000); Serial.print("\t");
    Serial.print(temp_c, 1);     Serial.print("\t");
```

Aniversario 75 UAT 1950-2025

```
    Serial.print(temp_f, 1);     Serial.print("\t");
    Serial.print(humedad, 1);    Serial.print("\t");
    Serial.print(heat_idx, 1);  Serial.print("\t");
    Serial.println(estado);
}
```

Practica 6 Actuadores — Motor DC y Servomotor

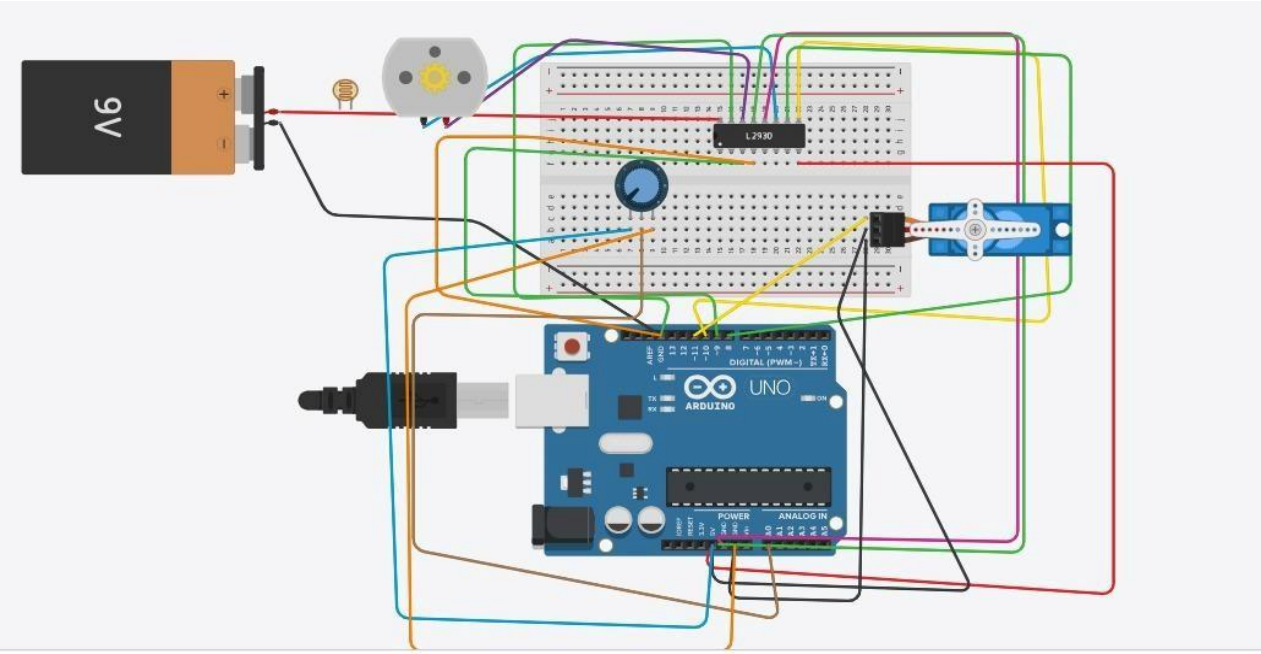
Materiales

Arduino UNO o Nano

- Motor DC 5-12V (pequeño)
- Driver L298N o L293D
- Servomotor SG90 o MG90S
- Potenciómetro 10 kΩ
- Fuente externa 5-9V/1A para motor (recomendado)
- Protoboard + cables dupont

Objetivo de la práctica

Comprender el funcionamiento y control de actuadores en sistemas embebidos mediante el uso de un motor de corriente directa (DC) y un servomotor en la plataforma Arduino Uno, con el fin de generar movimiento controlado, regular velocidad y posición, y aplicar estos dispositivos en la automatización de procesos.



Codigo

```
#include <Servo.h>

//  MOTOR DC (L293D)
const int ENA = 10; // PWM velocidad
const int IN1 = 8;  // Dirección
const int IN2 = 9;  // Dirección

//  SERVO
Servo mi_servo;
const int PIN_SERVO = 11;
const int PIN_POT = A0;

int angulo_anterior = -1;
```

```
//      FUNCIONES MOTOR
void motorAdelante(int velocidad) {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, velocidad);
}

void motorAtras(int velocidad) {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  analogWrite(ENA, velocidad);
}

void motorFreno() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, HIGH);
  analogWrite(ENA, 255);
}

void motorLibre() {
  analogWrite(ENA, 0);
}

void setup() {
  // Motor
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);

  // Servo
  mi_servo.attach(PIN_SERVO);
  mi_servo.write(90);

  Serial.begin(9600);
  Serial.println("Sistema listo");
}

void loop() {

  //  MOTOR ADELANTE (velocidad creciente)
```

VERDAD, BELLEZA, PROBIDAD

```
for (int v = 100; v <= 255; v += 20) {
  motorAdelante(v);
  moverServo()
```

Practica 7 Comunicación I2C — Sensor BMP280 y Pantalla OLED

Materiales

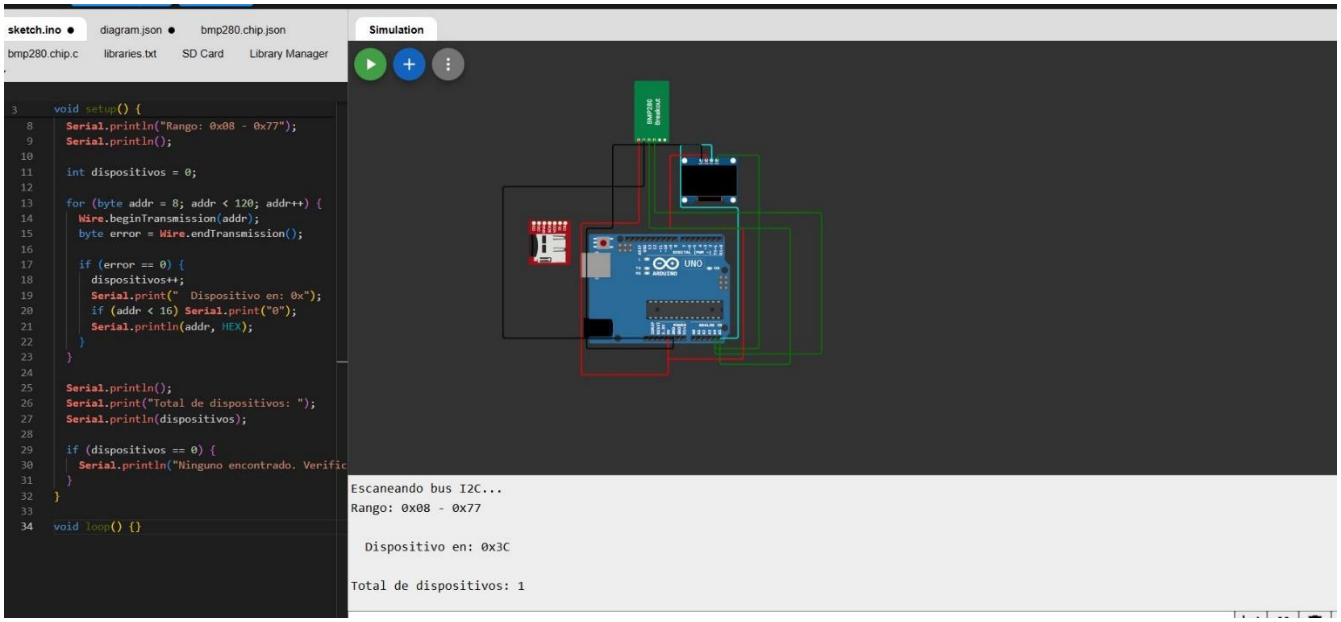
Arduino UNO o Nano

- Sensor BMP280 (módulo GY-BMP280)
- Pantalla OLED SSD1306 128×64 I2C
- Resistencias pull-up 4.7 kΩ × 2 (si no tiene las integradas)
- Protoboard + cables dupont

Objetivo de la práctica

Comprender el funcionamiento del protocolo de comunicación I2C en sistemas embebidos mediante la interconexión de dispositivos como el sensor BMP280 y una pantalla OLED en la plataforma Arduino Uno, con el fin de adquirir, transmitir y visualizar datos de manera eficiente utilizando un bus de comunicación compartido.

VERDAD, BELLEZA, PROBIDAD



Codigo

```
// Lab07_I2C_Scanner.ino
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);

  Serial.println("Escaneando bus I2C...");
  Serial.println("Rango: 0x08 - 0x77");
  Serial.println();

  int dispositivos = 0;

  for (byte addr = 8; addr < 120; addr++) {
    Wire.beginTransmission(addr);
    byte error = Wire.endTransmission();
```

```
    if (error == 0) {
      dispositivos++;
      Serial.print("  Dispositivo en: 0x");

      if (addr < 16) Serial.print("0"); // para formato 0x0X
      Serial.println(addr, HEX);
    }
  }

  Serial.println();
  Serial.print("Total de dispositivos: ");
  Serial.println(dispositivos);

  if (dispositivos == 0) {
    Serial.println("Ninguno encontrado. Verifica conexiones y pull-ups.");
  }
}

void loop() {
  // No se usa
}
```

Codigo

```
// Lab07_BMP280_OLED.ino

#include <Wire.h>
#include <Adafruit_BMP280.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_GFX.h>

#define OLED_W   128
#define OLED_H    64
#define OLED_ADDR 0x3C

Adafruit_BMP280 bmp;
Adafruit_SSD1306 display(OLED_W, OLED_H, &Wire, -1);

// Presión al nivel del mar (ajusta según tu ciudad)
const float PRESION_MAR = 1013.25; // hPa
```

```
void setup() {
  Serial.begin(9600);

  // ————— INICIAR OLED —————
  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("ERROR: OLED no encontrada en 0x3C");
    while (true);
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(20, 28);
  display.println("Iniciando BMP280...");
  display.display();

  // ————— INICIAR BMP280 —————
  if (!bmp.begin(0x76)) {
    Serial.println("ERROR: BMP280 no encontrado en 0x76");

    display.clearDisplay();
    display.setCursor(5, 28);
    display.println("ERROR: BMP280");
    display.display();

    while (true);
  }

  Serial.println("BMP280 y OLED listos.");
  Serial.println("Tiempo\tTemp(C)\tPresion(hPa)\tAltitud(m)");
}

void loop() {

  float temp     = bmp.readTemperature();      // °C
  float presion  = bmp.readPressure() / 100.0; // hPa
  float altitud  = bmp.readAltitude(PRESION_MAR);

  // ————— MOSTRAR EN OLED —————
  display.clearDisplay();
```

```
  // Título
  display.setTextSize(1);
  display.setCursor(30, 0);
  display.println("-- BMP280 --");

  // Temperatura grande
  display.setTextSize(2);
  display.setCursor(0, 14);
  display.print(temp, 1);
  display.println(" C");

  // Presión y altitud
  display.setTextSize(1);
  display.setCursor(0, 38);
  display.print("P: ");
  display.print(presion, 1);
  display.println(" hPa");

  display.setCursor(0, 50);
  display.print("Alt: ");
  display.print(altitud, 0);
  display.println(" m");

  display.display();

  // ————— MOSTRAR EN SERIAL —————
  Serial.print(millis() / 1000); Serial.print("\t");
  Serial.print(temp, 2);         Serial.print("\t");
  Serial.print(presion, 2);      Serial.print("\t");
  Serial.println(altitud, 1);

  delay(1000);
}
```

Practica 8 Comunicación SPI y Proyecto Integrador Bloque I

Materiales

Arduino UNO/Nano o ESP32 (recomendado para SPI)

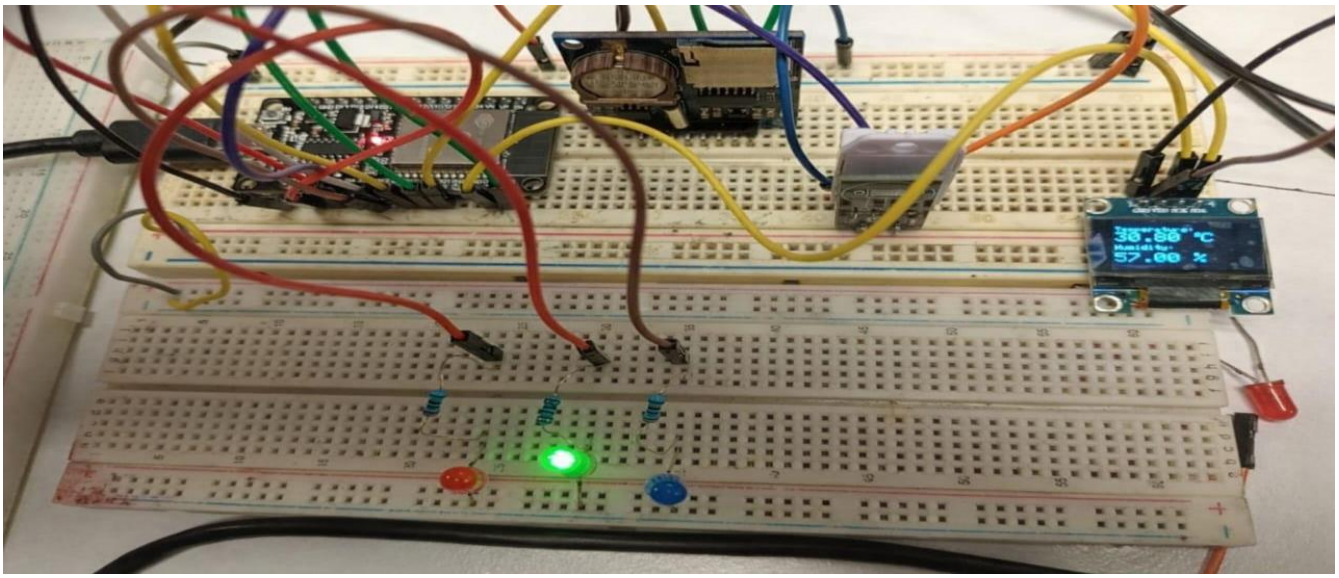
- Módulo lector tarjeta SD (SPI)
- Tarjeta microSD 2-32 GB
- Sensor DHT22 (del Lab 05)
- Pantalla OLED SSD1306 I2C (del Lab 07)
- LED RGB (o 3 LEDs: R/G/B) + resistencias
- Pulsador + resistencia 10kΩ

Objetivo de la práctica

Comprender el funcionamiento del protocolo de comunicación SPI en sistemas embebidos mediante la conexión e intercambio de datos entre múltiples dispositivos en la plataforma Arduino Uno, así como integrar los conocimientos adquiridos a lo largo del bloque en el desarrollo de un proyecto funcional, permitiendo el diseño e implementación de un sistema completo que involucre sensores, actuadores y comunicación digital.

VERDAD, BELLEZA, PROBIDAD

Aniversario 75 UAT 1950-2025



Codigo

```
from machine import Pin, I2C, SPI
import machine
import ssd1306
import time
import dht
import os
import sdcard  #  IMPORTANTE

# - - - - - - - - - - - - - - -
# OLED (I2C)
# - - - - - - - - - - - - - - -
i2c = I2C(0, sda=Pin(21), scl=Pin(22))
```

```
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

# ----------------
# LEDs (ajusta si cambiaste pines)
# ----------------
led_rojo = Pin(15, Pin.OUT)
led_verde = Pin(2, Pin.OUT)
led_azul = Pin(17, Pin.OUT)

def set_led(r, g, b):
    led_rojo.value(r)
    led_verde.value(g)
    led_azul.value(b)

# ----------------
# DHT22
# ----------------
sensor = dht.DHT22(Pin(4))

# ----------------
# SPI + SD (CORREGIDO)
# ----------------
spi = SPI(1, baudrate=10000000, pol…
[7:31 p.m., 13/4/2026] Mi Numero: #include <Wire.h>
#include <SPI.h>
#include <SD.h>
#include <DHT.h>
#include <Adafruit_SSD1306.h>

// —— Pines ESP32 ————————————————————————
#define PIN_DHT      4
#define PIN_CS_SD    5
#define PIN_BTN      2

#define PIN_LED_R    15
#define PIN_LED_G    2
#define PIN_LED_B    17

#define PIN_BUZZER  16

DHT dht(PIN_DHT, DHT22);
```

VERDAD, BELLEZA, PROBIDAD

```
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

// —— FSM ————————————————————————————————————
enum Estado { MONITOR, ALERTA, LOG };
Estado estado_actual = MONITOR;

// —— Variables ————————————————————————————
float temp_c = 0, humedad = 0;
unsigned long t_ultimo_guardado = 0;
unsigned long t_btn_presionado  = 0;
int total_registros = 0;

const int INTERVALO_GUARDADO = 10000;

// —— Funciones ————————————————————————————
void setLED(bool r, bool g, bool b) {
  digitalWrite(PIN_LED_R, r);
  digitalWrite(PIN_LED_G, g);
  digitalWrite(PIN_LED_B, b);
}

void guardarEnSD() {
  File f = SD.open("datos.csv", FILE_APPEND);

  if (f) {
    f.print(millis()/1000); f.print(",");
    f.print(temp_c,1);      f.print(",");
    f.println(humedad,1);
    f.close();

    total_registros++;
    Serial.print("Guardado #");
    Serial.println(total_registros);
  }
}

// —— SETUP ————————————————————————————————
void setup() {
  Serial.begin(115200);
```

VERDAD, BELLEZA, PROBIDAD

```
  pinMode(PIN_BTN, INPUT_PULLUP);
  pinMode(PIN_LED_R, OUTPUT);
  pinMode(PIN_LED_G, OUTPUT);
  pinMode(PIN_LED_B, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);

  dht.begin();

  // I2C ESP32
  Wire.begin(21, 22);

  // OLED
  if(!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("Error OLED");
    while(true);
  }

  oled.clearDisplay();
  oled.setTextColor(SSD1306_WHITE);

  // SD
  if (SD.begin(PIN_CS_SD)) {
    if (!SD.exists("datos.csv")) {
      File f = SD.open("datos.csv", FILE_WRITE);
      f.println("tiempo_s,temp_c,humedad_pct");
      f.close();
    }
    Serial.println("SD lista.");
  } else {
    Serial.println("Error SD");
  }

  Serial.println("Sistema iniciado. Estado: MONITOR");
}

// —— LOOP ————————————————————————————————————
void loop() {

  // Leer sensor
  float h = dht.readHumidity();
  float t = dht.readTemperature();
```

VERDAD, BELLEZA, PROBIDAD

```
if (!isnan(h) && !isnan(t)) {
  humedad = h;
  temp_c = t;
}

// Botón
bool btn = (digitalRead(PIN_BTN) == LOW);

if (btn) {
  if (t_btn_presionado == 0) t_btn_presionado = millis();
  if (millis() - t_btn_presionado > 3000) estado_actual = LOG;
} else {
  t_btn_presionado = 0;
  if (estado_actual == LOG) estado_actual = MONITOR;
}

// Alerta
if (temp_c > 35.0 || humedad > 85.0)
  estado_actual = ALERTA;
else if (estado_actual == ALERTA)
  estado_actual = MONITOR;

// OLED
oled.clearDisplay();
oled.setTextSize(1);

switch (estado_actual) {

  case MONITOR:
    setLED(0,1,0);
    oled.setCursor(0,0); oled.print("MONITOR");

    oled.setTextSize(2);
    oled.setCursor(0,16);
    oled.print(temp_c,1); oled.print("C");

    oled.setTextSize(1);
    oled.setCursor(0,40);
    oled.print("Hum: "); oled.print(humedad,1); oled.print("%");
```

VERDAD, BELLEZA, PROBIDAD

```
  oled.setCursor(0,54);
  oled.print("Reg: "); oled.print(total_registros);

  if (millis() - t_ultimo_guardado >= INTERVALO_GUARDADO) {
    guardarEnSD();
    t_ultimo_guardado = millis();
  }
break;

case ALERTA:
  setLED(1,0,0);
  tone(PIN_BUZZER, 1500, 300);

  oled.setCursor(20,0);
  oled.print("!!! ALERTA !!!");

  oled.setTextSize(2);
  oled.setCursor(0,16);
  oled.print(temp_c,1); oled.print("C");

  oled.setTextSize(1);
  oled.setCursor(0,42);

  if (temp_c > 35) oled.print("TEMP ALTA");
if (humedad > 85) oled.print("HUM ALTA");
break;

case LOG:
  setLED(0,0,1);

  oled.setCursor(10,10);
  oled.print("MODO REGISTRO");

  oled.setCursor(10,28);
  oled.print("Total:");

  oled.setTextSize(2);
  oled.setCursor(10,40);
  oled.print(total_registros);

  oled.setTextSize(1);
```

VERDAD, BELLEZA, PROBIDAD

```
    oled.setCursor(0,58);
   oled.print("Suelta boton");
   break;
  }

  oled.display();
  delay(500);
}
```

Código

```
from machine import Pin, I2C, SPI import machine
import ssd1306
import time import
dht import os
import sdcard          #   IMPORTANTE

# ----------------
# OLED (I2C) #
i2c = I2C(0, sda=Pin(21), scl=Pin(22))
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

# ----------------
# LEDs (ajusta si cambiaste pines) #
led_rojo     =     Pin(15,     Pin.OUT)
led_verde = Pin(2, Pin.OUT) led_azul
= Pin(17, Pin.OUT)

def set_led(r, g, b):
    led_rojo.value(r)
```

VERDAD, BELLEZA, PROBIDAD

```
    led_verde.value(g) led_azul.value(b)

# ----------------
# DHT22 #
sensor = dht.DHT22(Pin(4))
----------------

# ----------------
# SPI + SD (CORREGIDO) #
spi = SPI(1, baudrate=10000000, polarity=0, phase=0, sck=Pin(18), mosi=Pin(23),
----------------
          miso=Pin(19))

cs = Pin(5, Pin.OUT) sd_ok =

False

try:
    sd = sdcard.SDCard(spi, cs)              # CAMBIO CLAVE
    os.mount(sd, "/sd")
    print("SD OK") sd_ok =
    True
except Exception as e:
    print("Error SD:", e)

# ----------------
# FSM (Estados) #
IDLE = 0 ----------
```

VERDAD, BELLEZA, PROBIDAD

```
READ = 1
DISPLAY = 2
SAVE = 3
ERROR = 4

estado = IDLE temp =

0
hum = 0

# - - - - - - - - - - - - - - - - -
# FUNCIONES #
def leer_sensor(): global
- - - - - - - - - - - - - - - - -
    temp, hum try:
        sensor.measure()
        temp = sensor.temperature() hum =
        sensor.humidity() return True
    except:
        return False

def mostrar_oled(msg1, msg2=""): oled.fill(0)
    oled.text(msg1, 0, 0)
    oled.text(msg2, 0, 16) oled.show()

def guardar_sd():
```

VERDAD, BELLEZA, PROBIDAD

```
    try:
        with open("/sd/datos.txt", "a") as f:
            f.write("Temp: {:.1f}C Hum: {:.1f}%\n".format(temp, hum)) return True
    except Exception as e: print("Error
        guardando:", e) return False


# _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
# LOOP PRINCIPAL (FSM) #
while True: _ _ _ _ _ _ _ _ _ _ _ _

    if estado == IDLE: set_led(0, 1, 0)  #
        verde
        mostrar_oled("Sistema listo") time.sleep(2)
        estado = READ

    elif estado == READ: set_led(0,
        0, 1)  # azul if leer_sensor():
            estado = DISPLAY else:
            estado = ERROR


    elif estado == DISPLAY: set_led(1, 1, 0)  #
        amarillo
        mostrar_oled("Temp: {:.1f}C".format(temp), "Hum:
                    {:.1f}%".format(hum))
```

VERDAD, BELLEZA, PROBIDAD

```
    time.sleep(2) estado =
    SAVE

elif estado == SAVE: set_led(1, 0, 1)
    # morado if sd_ok and
    guardar_sd():
        mostrar_oled("Guardado OK") else:
        mostrar_oled("Error SD") estado
        = ERROR continue
    time.sleep(2) estado =
    IDLE

elif estado == ERROR: set_led(1, 0, 0)  #
    rojo mostrar_oled("ERROR")
    time.sleep(3)
    estado = IDLE
```

VERDAD, BELLEZA, PROBIDAD

Conclusión

A lo largo del desarrollo de las prácticas de laboratorio, se logró adquirir una comprensión sólida de los principios fundamentales de los sistemas embebidos mediante el uso de la plataforma Arduino Uno. Desde el control básico de salidas digitales hasta la implementación de protocolos de comunicación como I2C y SPI, cada actividad permitió fortalecer habilidades tanto teóricas como prácticas en electrónica y programación.

Asimismo, se comprendió la importancia de la correcta interacción entre sensores y actuadores, permitiendo la adquisición de datos del entorno y la ejecución de acciones en función de estos. El uso de dispositivos como pulsadores, sensores analógicos, motores y pantallas facilitó el entendimiento de cómo se integran los distintos componentes en un sistema funcional.

Finalmente, el desarrollo del proyecto integrador permitió aplicar de manera conjunta todos los conocimientos adquiridos, demostrando la capacidad de diseñar e implementar soluciones tecnológicas orientadas a la automatización y el monitoreo. Estas prácticas constituyen una base esencial para el desarrollo de proyectos más avanzados en el área de los sistemas embebidos.